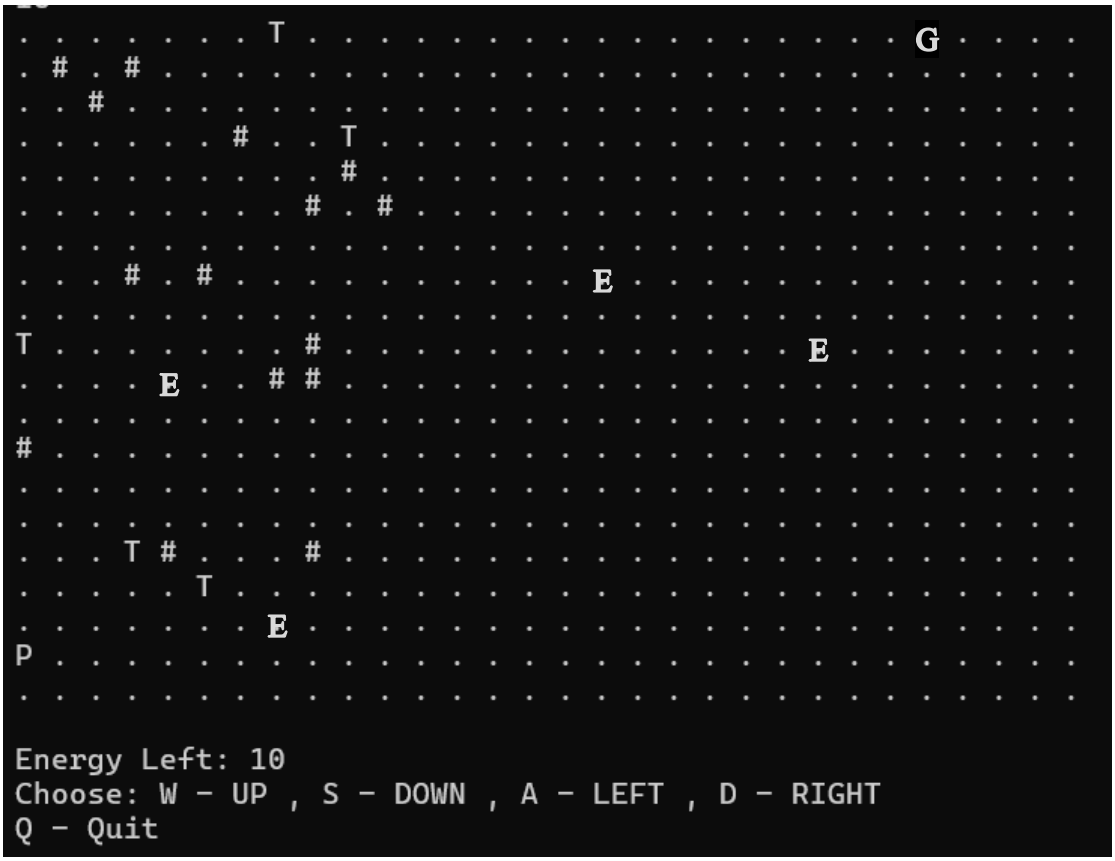
Practical 8 (due 2026-04-24 @ 09:00)

The purpose of this assignment is to familiarise you with two-dimensional dynamic arrays and modules.

Please note: Submissions will be checked for originality. If you use someone else's code or get code from the Internet, your submission will be investigated as a potential copy, which may result in zero marks being awarded along with potential further disciplinary action.

GRID NAVIGATOR

You have been tasked with creating a two-dimensional turn-based simulation to navigate a game world while managing energy and avoiding obstacles. The aim is for the human-controlled character to move through the game world and reach the goal location before running out of energy.



Example: *Player (P), Immovable Obstacles (#), Traps (T), Goal (G), Energy Pickups (E), Space (dots)*
NB: *Yours will not look exactly like this.*

Player (P), Immovable Obstacles (#), Traps (T), Energy Pickups (E), Goal (G), Space (dots) **NB: Yours will not look exactly like this.**

The player can move up (**W**), down (**S**), left (**A**) or right (**D**) at each turn and **cannot move diagonally**.

The player starts with a **user-defined number of units of energy** and loses **1 (one) unit of energy after each turn**.

When the player moves over an **energy pickup (E)**, the pickup disappears at that location and is replaced by the player, and the player's energy units are incremented by **5 units**.

When the player moves over a **trap (T)**, the trap disappears at that location and is replaced by the player, and the player's energy units are reduced by **3 units**.

The player cannot occupy the same space as an **immovable obstacle (#)**.

The game ends in **victory** when the player reaches the **goal location (G)** before running out of energy. The game ends in **defeat** when the player runs out of energy.

Please Note:

- You must use a module called `libGame` and place the logic of the game in the `NavigatorSpace` **`namespace`**. **(Do not use a header file)**
- **If using the VPL**, you must create an `input.txt` file that will contain your command-line arguments

Initialisation:

- The size of the environment (rows and columns), number of initial energy units (greater than 1), number of obstacles, number of traps, and number of energy pickups are specified as command-line arguments.
- The game world must be populated in the following order. Multiple items cannot occupy the exact location in the world:
  - Space
  - Immovable obstacles must be scattered randomly across the world.
  - Traps must be scattered randomly across the world.
  - Energy pickups must be scattered randomly across the world.
  - The goal location must be placed randomly in the **top row**
  - The human-controlled player starts at a random location in the **bottom row**

Movement:

- The player may move one step in any direction, except diagonally.
- The player may not move outside the game world.
- The player may move to any location in the game world area if an immovable obstacle does not occupy it.
- With every turn (**movement or attempted movement into an immovable object**), the player loses **1 unit of energy**.
- If the player moves over an **energy pickup**, the player gains **5 units of energy**.
- If the player moves over a **trap**, the player loses **3 additional units of energy.**
- If the player moves onto the **goal location**, the game ends in victory.

End of Game:

- The game ends in victory when the player reaches the **goal location**.
- The game ends in defeat when the player runs out of energy.

The entire design, including the UML Activity Diagram, must model the player movement logic. Any submission that does not compile will be capped to 40%.

Mark sheet		
	Program design (Movement logic)	10
	Coding style	10
	Memory allocation	10
	Memory deallocation	10
	Initialisation	10
	Use of Module	10
	User interaction	10
	Command line argument handling	10
	Algorithm implementation	20
		/100